

6. Ejemplos prácticos de conexión a APIs públicas

Las **APIs públicas** permiten acceder a datos sin necesidad de configurar un servidor propio. Muchas devuelven **JSON**, ideal para practicar AJAX.

6.1 Ejemplo 1: Consumir API de usuarios aleatorios

- **API:** [Random User API](#)
- Devuelve datos de usuario en JSON.

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX API Pública</title>
</head>
<body>
  <h1>Usuarios aleatorios</h1>
  <button id="btnCargar">Cargar usuario</button>
  <div id="usuarios"></div>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript (script.js)

```
document.getElementById("btnCargar").addEventListener("click",
function() {
  fetch("https://randomuser.me/api/")
    .then(response => response.json())
    .then(data => {
      let usuario = data.results[0];
      let div = document.getElementById("usuarios");
      div.innerHTML = `
        <p>Nombre: ${usuario.name.first}
        ${usuario.name.last}</p>
        <p>Email: ${usuario.email}</p>
        
      `;
    })
    .catch(error => console.error("Error:", error));
});
```

Funcionamiento:

- Al hacer clic, se solicita un usuario aleatorio.
- Los datos se muestran dinámicamente en la página.
- Perfecto para practicar **JSON** y **manipulación del DOM**.

6.2 Ejemplo 2: Consumir API de Pokémon

- API: [PokéAPI](#)
- Devuelve información de Pokémon en JSON.

JavaScript

```
fetch("https://pokeapi.co/api/v2/pokemon/1/")
  .then(response => response.json())
  .then(pokemon => {
    console.log(pokemon);
    document.body.innerHTML += `
      <h2>${pokemon.name.toUpperCase()}</h2>
      
    `;
  });
```

Funcionamiento:

- Muestra el nombre y la imagen de un Pokémon.
- Permite ver cómo acceder a **propiedades anidadas en JSON**.

6.3 Ejemplo 3: Consumir API de Clima

- API pública gratuita: [OpenWeatherMap](#)
- Necesita API key gratuita.

JavaScript

```
const apiKey = "TU_API_KEY"; // Regístrate para obtenerla
const ciudad = "Madrid";

fetch(`https://api.openweathermap.org/data/2.5/weather?q=${ciudad}&appid=${apiKey}&units=metric&lang=es`)
  .then(response => response.json())
  .then(data => {
    document.body.innerHTML += `
      <h2>Clima en ${data.name}</h2>
      <p>Temperatura: ${data.main.temp}°C</p>
      <p>Descripción: ${data.weather[0].description}</p>
    `;
  })
  .catch(error => console.error(error));
```

Funcionamiento:

- Se obtiene información del clima en tiempo real.
- Los datos JSON se procesan y muestran en HTML dinámicamente.

6.4 Buenas prácticas al consumir APIs públicas

1. **Siempre manejar errores** con `.catch()` o `onerror`.
2. **Revisar documentación** de la API (rutas, parámetros, límites).
3. **No exponer claves privadas** (usar backend si es necesario).
4. **Optimizar peticiones** evitando solicitudes innecesarias.

6.5 Ejercicios propuestos

1. Usar la **Random User API** para mostrar **5 usuarios** en una lista en la página.
2. Crear un buscador de Pokémon que consulte la API de PokéAPI según el número o nombre ingresado por el usuario.
3. Usar la API de OpenWeatherMap para mostrar el clima de **tres ciudades diferentes** en tarjetas separadas.